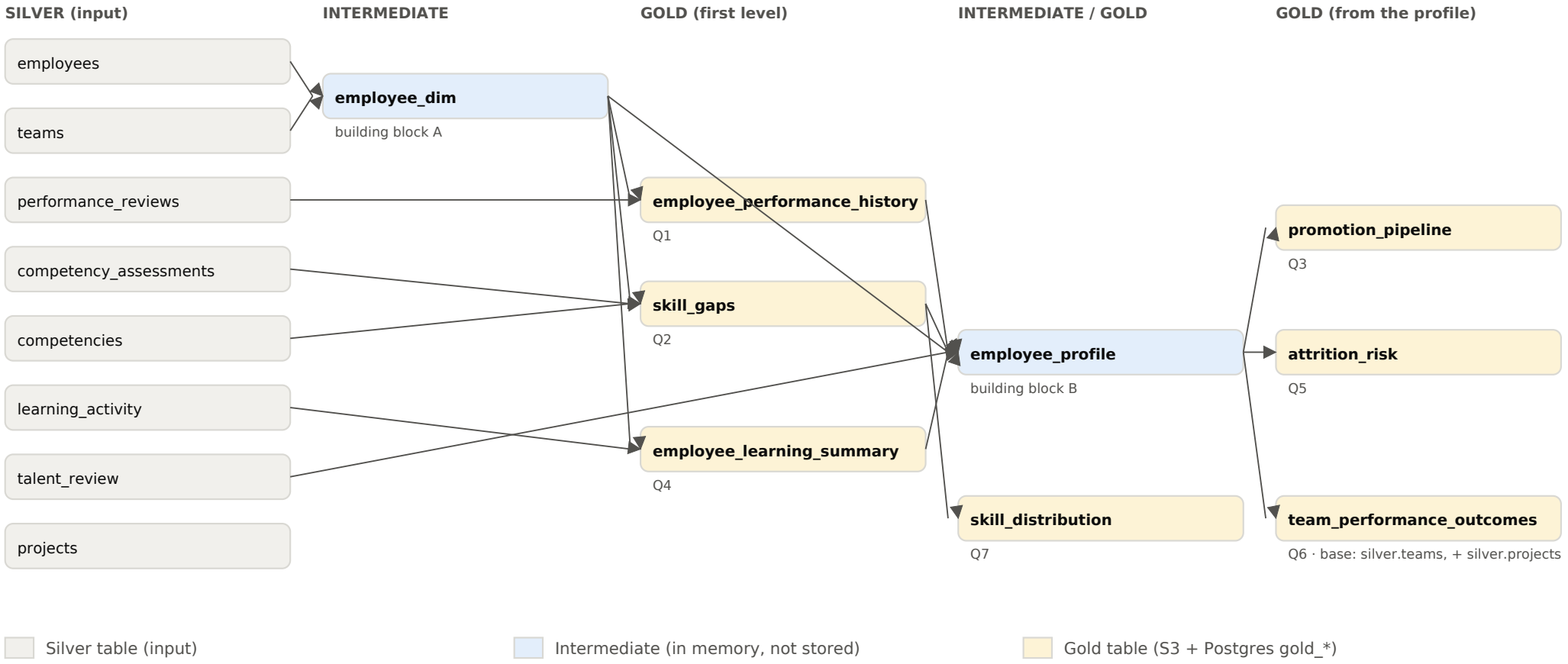
Gold tables: joins and column lineage

ACME talent pipeline · how each Gold table is built from the Silver layer · 15 October 2026

Every Gold table is built from **Silver** tables (validated, de-duplicated data) by the pure builder functions in `job.py` section 13. Two **intermediate** DataFrames are built first and reused, so shared logic exists once: `employee_dim` (who each employee is) and `employee_profile` (one row of facts per employee). Intermediates live only in memory during a run; the seven Gold tables are written to S3 (`gold/<table>/`) and to Postgres (`gold_<table>`). Business-rule values (e.g. critical gap ≥ 2) are recorded in every run manifest.



Join summary

Which table is the base (left side), what it is joined with, on which key, and with which join type. **left** keeps every row of the base; **inner** keeps only matching rows (Silver foreign-key checks guarantee every key has a match, so inner joins drop nothing).

Output	Base (left side)	Joined with	Key	Type
`employee_dim`	`silver.employees`	`silver.teams`	`team_id`	left
`gold.employee_performance_history`	`silver.performance_reviews`	`employee_dim` (context columns)	`employee_id`	inner
`gold.skill_gaps`	`silver.competency_assessments`	`silver.competencies`	`competency_id`	inner
		`employee_dim` (context columns)	`employee_id`	inner
`gold.employee_learning_summary`	`employee_dim` (context columns)	`silver.learning_activity`, events ≤ run date, aggregated per employee	`employee_id`	left
`employee_profile`	`employee_dim`	latest review from `gold.employee_performance_history`	`employee_id`	left
		average rating from `gold.employee_performance_history`	`employee_id`	left
		`silver.talent_review`	`employee_id`	left
		critical-gap counts from `gold.skill_gaps`	`employee_id`	left
		learning facts from `gold.employee_learning_summary`	`employee_id`	left
`gold.promotion_pipeline`	`employee_profile`	(no join: columns derived from the profile)	–	–
`gold.attrition_risk`	`employee_profile`	(no join: columns derived from the profile)	–	–
`gold.team_performance_outcomes`	`silver.teams`	`employee_profile` aggregated per team	`team_id`	left
		`silver.projects` aggregated per team	`team_id`	left
`gold.skill_distribution`	`gold.skill_gaps`	(no join: group-by + window share)	–	–

Every Gold table also gets two lineage columns when written: `_generated_at` (timestamp) and `_run_id` (the pipeline run that produced it).

Building block A: employee_dim (intermediate)

Grain: **one row per employee** · built by `build_employee_dim` in `data/talent-pipeline/job.py`, section 13 (Gold)

Source table	Join (type · key)	Columns taken	Becomes / calculation
`silver.employees`	base	`employee_id`, `first_name`, `last_name`, `job_level`, `team_id`, `hire_date`	`full_name` = first_name + ' ' + last_name; `tenure_years` = (run date − hire_date) / 365.25
`silver.teams`	left · `team_id`	`name`, `department`	`name` → `team_name`; `department` kept

Most Gold tables take the 'employee context' from here: `employee_id`, `full_name`, `department`, `team_name`, `job_level`.

Q1 · gold.employee_performance_history

Grain: **one row per employee × review period** · built by `build_performance_history` in `data/talent-pipeline/job.py`, section 13 (Gold)

Source table	Join (type · key)	Columns taken	Becomes / calculation
`silver.performance_reviews`	base	`review_id`, `employee_id`, `review_period`, `period_year`, `period_half`, `rating`, `manager_comment`, `submitted_date`	kept as-is
`employee_dim`	inner · `employee_id`	`full_name`, `department`, `team_name`, `job_level`	kept as-is (context)

Calculated column	Logic
`review_seq`	row_number() over (partition by employee_id order by period_year, period_half)
`previous_rating`	lag(rating) over the same window: the employee's rating in the previous period
`rating_change`	rating − previous_rating
`rating_trend`	'first review' if no previous rating, else 'improved' (> 0), 'declined' (< 0), 'stable' (= 0)

Q2 · gold.skill_gaps

Grain: **one row per employee × competency** · built by `build_skill_gaps` in `data/talent-pipeline/job.py`, section 13 (Gold)

Source table	Join (type · key)	Columns taken	Becomes / calculation
`silver.competency_assessments`	base	`employee_id`, `competency_id`, `proficiency_level`, `required_level`, `assessed_date`	kept as-is
`silver.competencies`	inner · `competency_id`	`name`, `category`	`name` → `competency_name`; `category` kept
`employee_dim`	inner · `employee_id`	`full_name`, `department`, `team_name`, `job_level`	kept as-is (context)

Calculated column	Logic
`gap`	required_level − proficiency_level (positive = below what the role needs)
`gap_severity`	'critical' if gap ≥ CRITICAL_GAP_LEVELS (2), 'moderate' if gap ≥ 1, else 'none'
`is_critical_gap`	gap ≥ 2

Q4 · gold.employee_learning_summary

Grain: **one row per employee (every employee, zeros if no activity)** · built by `build_learning_summary` in `data/talent-pipeline/job.py`, section 13 (Gold)

Source table	Join (type · key)	Columns taken	Becomes / calculation
`employee_dim`	base	`employee_id`, `full_name`, `department`, `team_name`, `job_level`	kept as-is (every employee keeps a row)
`silver.learning_activity`	left · `employee_id` (after group-by; only events with event_day ≤ run date)	`event_type`, `course_name`, `score`, `event_day`	aggregated per employee (see below); counts filled with 0 when there is no activity

Calculated column	Logic
`total_events`	count(*)
`courses_engaged` / `courses_enrolled`	count distinct course_name (any event) / where event_type = 'enrollment'
`courses_completed` / `certifications_earned`	count distinct course_name where event_type = 'course_completed' / 'certification_earned'
`quiz_events` / `quizzes_passed`	count where event_type in (quiz_attempted, quiz_passed) / = 'quiz_passed'
`avg_quiz_score`	avg(score), 1 decimal
`first_activity_date` / `last_activity_date`	min / max of event_day
`completed_courses`	sorted distinct list of course_name where event_type = 'course_completed' (empty list if none)
`completion_rate`	courses_completed / courses_engaged (2 decimals; null if no courses)
`days_since_last_activity`	run date − last_activity_date

Building block B: employee_profile (intermediate)

Grain: **one row per employee** · built by `build_employee_profile` in `data/talent-pipeline/job.py`, section 13 (Gold)

Source table	Join (type · key)	Columns taken	Becomes / calculation
`employee_dim`	base	all columns	kept (identity, team, department, tenure)
`gold.employee_performance_history` (latest review)	left · `employee_id`	`review_period`, `rating`, `previous_rating`, `rating_change`, `rating_trend`	row_number() over (partition by employee_id order by period desc) = 1; `review_period` → `latest_period`, `rating` → `latest_rating`
`gold.employee_performance_history` (all reviews)	left · `employee_id`	`rating`	avg(rating) → `avg_rating` (2 dp); count(*) → `review_count`
`silver.talent_review`	left · `employee_id`	`review_cycle`, `potential_rating`, `promotion_readiness`, `attrition_risk`	`attrition_risk` → `hr_attrition_risk`; others kept (HR's own ratings, not calculated)
`gold.skill_gaps`	left · `employee_id`	`is_critical_gap`	count where critical → `critical_gap_count`; count(*) → `competencies_assessed` (0 if none)
`gold.employee_learning_summary`	left · `employee_id`	`courses_completed`, `certifications_earned`, `last_activity_date`, `days_since_last_activity`	kept as-is

Built once and reused by Q3, Q5 and Q6, so promotion, attrition and team results always agree with each other.

Q3 · gold.promotion_pipeline

Grain: **one row per employee** · built by `build_promotion_pipeline` in `data/talent-pipeline/job.py`, section 13 (Gold)

Source table	Join (type · key)	Columns taken	Becomes / calculation
`employee_profile`	base (no join)	`employee_id`, `full_name`, `department`, `team_name`, `job_level`, `tenure_years`, `potential_rating`, `promotion_readiness`, `latest_period`, `latest_rating`, `avg_rating`, `rating_trend`, `critical_gap_count`, `certifications_earned`	kept as-is

Calculated column	Logic
`is_high_potential`	potential_rating = 'High'
`is_promotion_candidate`	High potential AND promotion_readiness = 'Ready Now' AND latest_rating ≥ PROMOTION_MIN_RATING (4)
`pipeline_stage`	'Promote now' (candidate, 0 critical gaps) · 'Promote after closing skill gaps' (candidate) · 'Develop (high potential)' · 'Not in pipeline'

Q5 · gold.attrition_risk

Grain: **one row per employee** · built by `build_attrition_risk` in `data/talent-pipeline/job.py`, section 13 (Gold)

Source table	Join (type · key)	Columns taken	Becomes / calculation
`employee_profile`	base (no join)	`employee_id`, `full_name`, `department`, `team_name`, `job_level`, `tenure_years`, `hr_attrition_risk`, `latest_rating`, `previous_rating`, `rating_trend`, `days_since_last_activity`, `potential_rating`	kept as-is

Calculated column	Logic
`risk_score`	HR High +3 / Medium +1 · rating_trend = declined +2 · latest_rating ≤ 2 +1 · days_since_last_activity > 180 or no activity +1 (RISK_POINTS)
`risk_tier`	'High' if score ≥ 4, 'Medium' if ≥ 2, else 'Low'
`risk_factors`	list of the reasons that added points, e.g. 'Rating declined 4 -> 2'

Q6 · gold.team_performance_outcomes

Grain: **one row per team (all 100 teams)** · built by `build_team_outcomes` in `data/talent-pipeline/job.py`, section 13 (Gold)

Source table	Join (type · key)	Columns taken	Becomes / calculation
`silver.teams`	base	`team_id`, `name`, `department`	`name` → `team_name`
`employee_profile`	left · `team_id` (after group-by team)	`latest_rating`, `rating_change`, `critical_gap_count`, `hr_attrition_risk`	count → `headcount`; avg `latest_rating` → `avg_latest_rating`; share rating ≥ 4 → `high_performer_share`; avg → `avg_rating_change`, `avg_critical_gaps`; count HR High → `hr_high_risk_count`
`silver.projects`	left · `team_id` (after group-by team)	`outcome_score`, `outcome_status`	count → `project_count`; avg → `avg_outcome_score`; share 'on-time' → `on_time_share`; count at-risk/cancelled → `troubled_projects`

Calculated column	Logic
`has_projects`	project_count > 0 (counts filled with 0 for teams without projects)

Q7 · gold.skill_distribution

Grain: **one row per department × competency × proficiency level** · built by `build_skill_distribution` in `data/talent-pipeline/job.py`, section 13 (Gold)

Source table	Join (type · key)	Columns taken	Becomes / calculation
`gold.skill_gaps`	base (no join): group by `department`, `category`, `competency_name`, `proficiency_level`	`employee_id`, `is_critical_gap`	count distinct employee_id → `employees`; count critical → `critical_gaps`

Calculated column	Logic
`share_of_competency`	employees / sum(employees) over (partition by department, competency_name), 3 decimals